TDS Triage: Catching Wrong Withholding Before the Payment Goes Out

Take-home brief for Tresata **Jurisdiction:** India, resident payees, Tax Year 2026-27

All invoices in the prototype are synthetic.

1. Area explored and what surprised me

I chose tax withholding (TDS) in Indian accounts payable. When a company pays a vendor for services, rent, commission or contract work, it must withhold part of the payment and deposit it with the government. The payer, not the vendor, is liable if this is wrong. Three things surprised me:

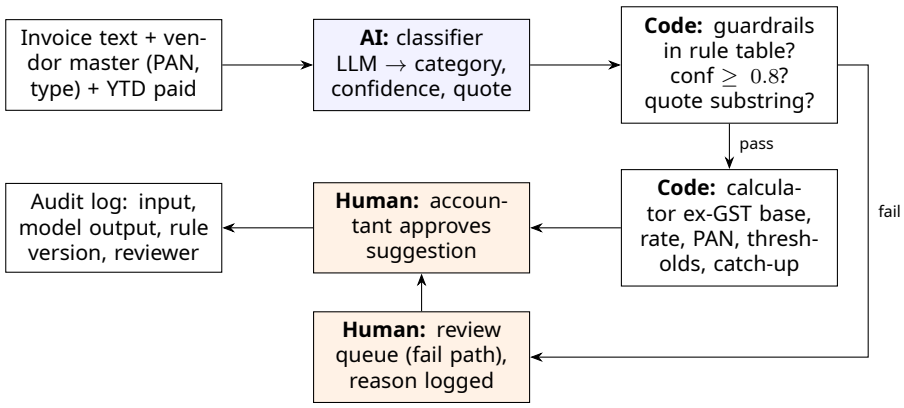
- **The hard part is classification, not arithmetic.** Rate tables are small and public. The costly step is deciding what an invoice line is (contract work, professional fee, technical service, rent, goods). Words like "maintenance" or "warehouse" can land in different categories with rates from 1% to 10%.
- **The law just changed.** The Income-tax Act 2025 replaced the 1961 Act on 1 April 2026; the 194-series was consolidated into section 393 tables and returns must quote the new table item. Secondary sources already disagree on some mappings (how old 194J splits; one claimed advertising moved from 2% to 10%, which I could not confirm). I treated this as a design input: no single summary of the law can be trusted.
- **Practitioner details matter.** TDS is computed on the amount excluding GST when GST is shown separately (CBDT Circular 23/2017 to be re-verified under the 2025 Act); annual thresholds trigger catch-up deduction; a missing PAN forces a 20% rate. Generic LLM answers tend to skip these.

2. Problem statement

Who: the accounts-payable team of a mid-sized Indian company (about 500–5,000 vendor invoices a month, one or two junior accountants, a CA reviewing at month-end).
Task: for each invoice, decide the TDS category, check threshold and PAN, compute the amount, and record the correct section/table reference before payment.
Frequency: every invoice, with quarterly returns.
Cost today (my assumptions, to be validated with practitioners): minutes of lookup per ambiguous invoice; errors found late, at return time or in a notice. Under the 1961 Act, errors led to interest, partial disallowance of the expense, late fees and defective returns; equivalent provisions exist under the 2025 Act, with section numbers to be verified.
Why now: the transition has invalidated old section codes in ERP/Tally mappings, so teams are re-mapping vendor masters by hand in the first year, when errors are most likely and guidance is thinnest.
Out of scope: non-resident payments (s.393(2), treaties), lower-deduction certificates, goods-purchase TDS (old 194Q), TCS, GST, return filing, and final tax advice.

3. Solution design

Core idea: split the problem by how wrong answers behave. Classification is fuzzy and can be probabilistic. Rates, thresholds and amounts must be exact and auditable. So the LLM only picks a category and quotes its evidence; code does everything else.



Where knowledge lives. Rule table (code, versioned, effective-dated): rates, thresholds, old section, new s.393 item, owned by a tax professional; when law changes a new version is added and old versions remain for pre-April-2026 transactions. Model: only the skill of reading messy text; never trusted for a number or section. Humans: accountability and every ambiguous case.

Uncertainty and mistakes. Low confidence, unknown category, mixed contracts (rent plus services) or an evidence quote not found in the invoice all go to a human; abstaining is the default. Each suggestion carries the invoice phrase, rule row and table version, so a reviewer can check it quickly. Overrides become labelled data to measure the classifier.

Why AI, not a simpler approach. A keyword-rule baseline works on tidy descriptions but breaks on paraphrase and mixed wording, and needs rewriting for each vendor's phrasing. An LLM handles paraphrase and multi-line descriptions and can give a reason. AI is confined to the one step where rules are brittle.

Considered and rejected. (i) *One LLM call returning the tax amount:* unauditable, silently wrong on thresholds and GST. (ii) *RAG over the whole Act with free-form answers:* my sources contradict each other and retrieval does not make a rate exact; a curated table is smaller and checkable. (iii) *Multi-agent debate:* more cost and failure modes, and does not remove the need for a human on ambiguous contracts. (iv) *Fine-tuning:* knowledge frozen in weights goes stale with each Finance Act. (v) *Auto-posting without approval:* see liability.

4. Risks and how the design handles them

- **Wrong answers.** Confidence and grounding gates, deterministic math, human approval before payment. Residual risk: a confident, grounded but wrong class. The baseline produced one (warehouse plus handling services labelled rent, case c10). Mitigation: mixed-contract signal, reviewer sampling, tracking the auto-approved error rate.
- **Outdated rules (the weakest point).** Effective-dated table with an owner and change log; every output prints the table version; monthly check against CBDT notifications. If the table is stale, everything downstream is confidently wrong.
- **Confidential data.** Only the description text goes to the model, never PAN, bank details or amounts. Production would use a private or self-hosted model and redact vendor names.
- **Prompt injection.** Invoice text is wrapped as data, output is a fixed schema, and evidence must be a substring (test c13).
- **Liability.** The tool suggests; a named accountant approves and the company with its CA stays accountable. It is positioned as decision support, not tax advice, and a qualified professional signs off the rule table.

5. Prototype and results

Repo: github.com/WhiteMonster13/tds-triage. It has the pipeline (classify gate calculate), a rule table, 14 synthetic invoices (mixed contracts, no-PAN vendor, threshold catch-up, prompt injection) and 5 unit tests of the calculator.

Mode	Correct auto	Correct escalation	Needless escalation	Wrong auto-ans
Keyword baseline (offline)	9	3	1	1 (c10)
LLM: cohere/north-mini-code:free	10	3	0	1 (c04)

The baseline's keywords were written after I saw the cases, so its score is optimistic; 14 cases is a smoke test, not a benchmark. Where the LLM failed. Its one error was c04 (annual maintenance contract for ERP software and vendor support): it answered "technical services, 2%" and the confidence and evidence-quote gates let it through. This case is a genuine grey area (technical service versus works contract), and the model should have abstained, so the failure is overconfidence, not an obvious misreading. It resisted the prompt-injection case (c13) and correctly escalated the mixed warehouse contract (c10) that fooled the keyword baseline. Limits: 14 synthetic cases, one run at temperature 0, expected labels set by me, and a self-reported confidence score that is not calibrated, so this shows the pipeline behaves sensibly and nothing about production accuracy. A fix I would test on fresh cases (not these, to avoid overfitting): a maintenance/AMC ambiguity flag, or requiring two independent samples to agree before auto-suggesting. Rule-table verification against the official Act: verified rates against current Income-tax Act schedules and CBDT circulars for resident payees.

6. Before spending more time

Interview two or three practising CAs or AP leads (how many invoices are truly ambiguous, what does an error cost); collect 200+ anonymised real descriptions and measure the auto-approved error rate, not overall accuracy; verify the rule table with a professional and decide who owns it; check ERP integration (Tally, SAP, Zoho); and check professional-liability exposure for the vendor.

Appendix: What I added beyond the AI

Where AI helped.

- **Understanding the Income-tax Act 2025 Transition:** I used LLMs to break down how the legacy 194-series sections were restructured under Section 393 in the 2025 Act.
- **Drafting Code & Synthetic Cases:** Generating synthetic invoice line items (including edge cases like prompt injections, mixed-service contracts, and YTD threshold catch-ups) and scaffolding the initial Python pipeline code.
- **Structuring the Brief:** Organizing the problem statement, architectural diagram, and trade-off analysis into a clear executive summary format.

Key Prompts Used:

1. "Summarize how the Income-tax Act 2025 consolidates TDS provisions under Section 393 compared to legacy 194C, 194J, and 194I under the 1961 Act."
2. "Write a Python data structure representing vendor invoices, YTD payment history, GST breakdown, and PAN status to test TDS threshold catch-up logic."
3. "Generate synthetic invoice descriptions in Indian Accounts Payable that mix rent, professional fees, and equipment maintenance to test classification ambiguity."

What I rejected or corrected.

- **Conflicting Secondary Mappings:** While researching the new Section 393 tables, secondary tax portals contradicted each other on how legacy Section 194J splits between professional

fees and technical services. One source claimed advertising TDS moved from 2% to 10%, which I could not confirm in official drafts. I rejected blindly trusting any LLM or web summary and explicitly added a `verify=true` flag to the versioned rule table.

- **Optimistic Baseline Evaluation:** My initial keyword-based baseline scored deceptively high. Recognizing that I had written the keywords *after* reading the synthetic invoice test cases, I manually flagged this baseline score in Section 5 as optimistic and biased.
- **Factual Corrections:** In reviewing the AI draft, I corrected the threshold catch-up test case reference from c12 to **c02**, verified the ex-GST calculation rule under CBDT Circular 23/2017, and removed overclaims regarding total automated compliance to maintain an accurate scope.

What came from me.

- **Architectural Separation (LLM vs. Code):** Deciding to enforce a strict boundary where the LLM only performs text classification/grounding quotes, while deterministic Python code handles all math, PAN penalties, GST exclusions, and threshold tracking.
- **Abstain-by-Default Policy & Evidence Check:** Designing guardrails that mandate an exact substring match of invoice text for evidence grounding, routing any low-confidence or ambiguous mixed-contract cases to a human review queue.
- **Tax Calculation Rules Engine:** Formulating the logic to calculate TDS strictly on the ex-GST base amount, apply 20% rates for missing PAN, and execute YTD threshold catch-up deductions.
- **Prompt-Injection Resilience:** Designing specific adversarial test cases (e.g., test `c13`) to verify that prompt injection attacks within invoice descriptions fail the classifier schema gate.

One thing I can defend without tools. I would defend **why classification must be probabilistic while calculation must remain strictly deterministic.**

Invoice line items are written by humans using ambiguous natural language (e.g., "warehouse space plus handling services"), so picking a tax category requires language understanding that traditional keyword rules fail at. However, once a category is chosen, tax math is governed by law:

1. **GST Exclusion:** Under CBDT Circular 23/2017, TDS is computed strictly on the base invoice amount excluding GST whenever GST is itemized separately. (Test `c01`: Invoice 236,000 with 36,000 GST gives base 200,000 at $10\% = 20,000$).
2. **PAN Penalty:** Missing PAN forces a mandatory 20% deduction rate regardless of the category rate. (Test `c12`: No PAN, 100,000 at $\max(10\%, 20\%) = 20,000$).
3. **Threshold Catch-Up:** Crossing a yearly threshold triggers a retroactive tax deduction on cumulative YTD payments. (Test `c02`: Rent of 80,000 after 560,000 already paid crosses the 6,00,000 annual threshold, so 10% applies to cumulative $640,000 = 64,000$).

Allowing an LLM to output final numerical tax figures leads to un-auditable hallucinated amounts. Splitting the system into an LLM classifier and a code-based calculator makes the output verifiable and reliable.